

Family Health Agent Implementation Plan v1.0

- Family Health Agent 工程落地任务书 v1.0
 - 0. 本任务书的使用方式
 - 1. 项目最终目标
 - 2. 总开发原则
 - 3. 项目目录不可变边界
 - 4. 总体开发阶段
- Phase 00: 仓库与工程规范
 - 目标
 - Task 00.1 确认目录结构
 - Task 00.2 建立工程规范文档
 - Task 00.3 创建基础 README
 - 验收标准
- Phase 01: 后端基础设施
 - 目标
 - Task 01.1 初始化 FastAPI 后端
 - Task 01.2 配置 Settings
 - Task 01.3 配置 SQLAlchemy
 - Task 01.4 配置 Alembic
 - Task 01.5 配置 Docker Compose
 - 验收标准
- Phase 02: 数据库模型与 Alembic
 - 目标
 - 通用建模规则
 - Task 02.1 identity 模块模型
 - Task 02.2 family 模块模型
 - Task 02.3 permissions 模块模型
 - Task 02.4 health_profile 模块模型
 - Task 02.5 health_data 模块模型
 - Task 02.6 health_record 模块模型
 - Task 02.7 medical_timeline 模块模型
 - Task 02.8 document_center / document_processing 模型
 - Task 02.9 reports / alerts 模型
 - Task 02.10 agent / audit 模型

- 验收标准
- Phase 03: Seed Demo Data
 - 目标
 - Task 03.1 创建 demo 用户
 - Task 03.2 创建 demo 家庭
 - Task 03.3 创建权限
 - Task 03.4 创建健康数据
 - Task 03.5 seed 可重复执行
 - 验收标准
- Phase 04: 核心业务模块 Service 层
 - 目标
 - 通用规则
 - Task 04.1 identity_service
 - Task 04.2 family_service
 - Task 04.3 permission_service
 - Task 04.4 health_data_service
 - Task 04.5 health_record_service
 - Task 04.6 medical_timeline_service
 - Task 04.7 document_center_service
 - Task 04.8 reports_service
 - Task 04.9 alerts_service
 - 验收标准
- Phase 05: 基础 API 层
 - 目标
 - Task 05.1 API Router
 - Task 05.2 Auth API
 - Task 05.3 Family API
 - Task 05.4 Health Data API
 - Task 05.5 Health Records API
 - Task 05.6 Reports / Alerts API
 - 验收标准
- Phase 06: 权限系统闭环
 - 目标
 - Task 06.1 权限检查测试
 - Task 06.2 API 权限接入
 - 验收标准
- Phase 07: Health Agent Harness
 - 目标

- Task 07.1 runtime.py
 - Task 07.2 permissions.py
 - Task 07.3 tool_registry.py
 - Task 07.4 safety.py
 - Task 07.5 tracing.py
 - Task 07.6 errors.py
 - Task 07.7 guards.py
 - Task 07.8 runner.py / tool_caller.py
 - 验收标准
- Phase 08: Agent Tools
 - 目标
 - Tools 清单
 - 文件路径
 - 工具规则
 - 验收标准
- Phase 09: LLM Client 与 Prompt 管理
 - 目标
 - Task 09.1 LLM Client
 - Task 09.2 Prompt 管理
 - 验收标准
- Phase 10: LangGraph Workflows
 - 目标
 - Task 10.1 chat_workflow
 - Task 10.2 free_text_record_workflow
 - Task 10.3 daily_report_workflow
 - Task 10.4 document_extract_workflow
 - Task 10.5 doctor_visit_summary_workflow
 - 验收标准
- Phase 11: 日报与提醒系统
 - 目标
 - Task 11.1 规则服务
 - Task 11.2 日报生成器
 - Task 11.3 提醒生成
 - 验收标准
- Phase 12: 文档中心与资料处理
 - 目标
 - 实现顺序
 - Task 12.1 文件上传与存储

- Task 12.2 文档处理任务
 - Task 12.3 OCR 适配器
 - Task 12.4 提取器
 - 验收标准
- Phase 13: 知识库 / RAG
 - 目标
 - 实现
 - 路径
 - 验收标准
- Phase 14: 导出与分享
 - 目标
 - 实现
 - 路径
 - 验收标准
- Phase 15: Web 前端
 - 目标
 - 页面顺序
 - 页面模块路径
 - 验收标准
- Phase 16: 测试体系
 - 目标
 - 测试目录
 - 必须测试
 - 关键测试用例
 - 验收标准
- Phase 17: 部署与运维
 - 目标
 - 实现
 - 验收标准
- Phase 18: 移动端 / 设备 / 通知扩展
 - 目标
 - 移动端
 - 设备接入
 - 通知
 - 验收标准
- 最终完成标准
- 给 Codex 的总启动 Prompt

- 每个 Phase 完成后的固定汇报格式

Family Health Agent 工程落地任务书 v1.0

适用对象：Codex / 开发者 / 项目负责人

项目：family-health-agent

架构原则：成品级模块化单体架构，阶段性点亮功能

核心定位：长期健康档案 + 家庭健康共享 + AI 健康管家

医疗边界：生活健康管理与风险提醒，不做医学诊断、不做处方建议、不替代医生

0. 本任务书的使用方式

这不是普通功能清单，而是整个项目的工程执行手册。开发时必须从上往下执行，不允许这里做一点、那里做一点。

正确使用方式：

1. 先阅读 docs/architecture/家庭健康Agent_项目架构设计_v1.0.md
2. 再阅读本文件 CODEX_IMPLEMENTATION_PLAN.md
3. 按 Phase 顺序执行
4. 每个 Phase 完成后先跑验收
5. 验收通过后再进入下一个 Phase
6. 不允许提前实现后续阶段

核心目标：

产品第一阶段可以只开放核心闭环；
但代码架构、目录边界、数据库地基、Agent 安全体系必须按最终成品标准建设。

1. 项目最终目标

最终要落地的是一个成品级家庭健康 Agent 系统，包含：

用户系统
家庭系统
权限系统
长期健康档案
健康指标记录
血压记录
症状/健康随手记
医疗事件时间线
原始资料中心
资料 AI 提取
日报/周报
提醒系统
AI 管家

LangGraph 工作流
Health Agent Harness
设备接入预留
通知系统预留
健康知识库/RAG 预留
导出与分享预留
审计与可追溯
Web 前端
移动端预留
后台管理预留

架构目标：

Monorepo
+ 模块化单体 Backend
+ 多前端入口 apps
+ Agent Core 独立域
+ 业务 modules 按领域拆分
+ integrations/jobs/workers 为未来设备、OCR、通知、队列预留
+ docs/prompts/datasets 作为长期工程资产管理

2. 总开发原则

2.1 绝对禁止事项

开发者和 Codex 不允许做以下事情：

不允许重构顶层目录结构。
不允许把业务逻辑写进 API 层。
不允许把所有 models/services/schemas 堆在根目录一坨。
不允许 Agent 直接访问数据库。
不允许 Workflow 直接创建数据库 session。
不允许 Tool 自己决定查询谁。
不允许 LLM 自由决定 user_id / family_id / target_user_id。
不允许 LLM 调用写入工具。
不允许未确认的 AI 草稿直接入库。
不允许跳过权限检查访问家人数据。
不允许把“系统无记录”说成“现实没有问题”。
不允许生成医学诊断、处方建议、药物剂量建议。
不允许为了赶进度破坏模块边界。
不允许在测试里依赖真实 LLM API。
不允许 document 工具返回真实文件路径给 LLM。

2.2 必须遵守事项

API 层只负责接收请求、校验参数、调用 service / agent_service。
Repository 只负责数据库读写，不写业务决策。
Service 负责业务逻辑，不依赖 LangGraph。
Agent Tools 只能调用 service，不直接查库。
LangGraph 只负责编排流程。
Harness 负责 runtime、权限、工具门禁、安全、追踪、错误处理。
规则 / Python 负责事实计算。
LLM 只负责意图识别、结构化草稿、自然语言表达。
所有健康输出必须过 Safety。

所有关键 Agent 执行必须写 Trace。
所有写入健康档案的动作必须有用户确认。
所有家人数据访问必须经过 family_id + permission check。

3. 项目目录不可变边界

项目根目录必须保持：

```
family-health-agent/  
├─ apps/  
├─ backend/  
├─ packages/  
├─ infra/  
├─ docs/  
├─ prompts/  
├─ datasets/  
├─ tools/  
├─ docker-compose.yml  
├─ docker-compose.dev.yml  
├─ .env.example  
├─ README.md  
└─ Makefile
```

后端核心结构必须保持：

```
backend/  
├─ app/  
│   └─ main.py  
│   └─ core/  
│   └─ db/  
│   └─ api/  
│   └─ modules/  
│   └─ agent/  
│   └─ integrations/  
│   └─ jobs/  
│   └─ workers/  
│   └─ common/  
│   └─ utils/  
├─ alembic/  
├─ tests/  
├─ scripts/  
├─ storage/  
├─ pyproject.toml  
├─ Dockerfile  
└─ README.md
```

业务模块必须放在：

```
backend/app/modules/<domain>/
```

Agent 代码必须放在：

```
backend/app/agent/
```

4. 总体开发阶段

必须按顺序执行：

Phase 00: 仓库与工程规范
Phase 01: 后端基础设施
Phase 02: 数据库模型与 Alembic
Phase 03: Seed Demo Data
Phase 04: 核心业务模块 Service 层
Phase 05: 基础 API 层
Phase 06: 权限系统闭环
Phase 07: Health Agent Harness
Phase 08: Agent Tools
Phase 09: LLM Client 与 Prompt 管理
Phase 10: LangGraph Workflows
Phase 11: 日报与提醒系统
Phase 12: 文档中心与资料处理
Phase 13: 知识库 / RAG
Phase 14: 导出与分享
Phase 15: Web 前端
Phase 16: 测试体系
Phase 17: 部署与运维
Phase 18: 移动端 / 设备 / 通知扩展

Phase 00: 仓库与工程规范

目标

让项目从第一天就有稳定工程规范，避免后期变成豆腐渣工程。

Task 00.1 确认目录结构

检查并补齐以下目录：

```
apps/web
apps/mobile
apps/admin
apps/miniapp
backend/app
backend/alembic
backend/tests
backend/scripts
packages/shared-types
packages/ui-kit
packages/api-client
infra/docker
infra/nginx
infra/postgres
infra/redis
infra/minio
infra/observability
docs/product
docs/architecture
docs/database
docs/api
docs/agent
docs/operations
docs/decisions
prompts/health_record
prompts/daily_report
prompts/document_extract
prompts/doctor_visit
prompts/safety
datasets/seed
datasets/demo
datasets/fixtures
tools/dev
tools/migration
tools/validation
```

Task 00.2 建立工程规范文档

创建：

```
docs/architecture/CODE_STYLE.md
docs/architecture/DEVELOPMENT_RULES.md
docs/architecture/MODULE_BOUNDARIES.md
docs/architecture/NO_GO_RULES.md
docs/operations/GIT_WORKFLOW.md
```

每份文档最低要求：

```
CODE_STYLE.md:
- Python 格式化工具
- TypeScript 格式化工具
- 命名规范
- import 规范

DEVELOPMENT_RULES.md:
- API / Service / Repository / Agent 分层规则
- 数据库事务规则
- 异步任务规则
- 测试要求

MODULE_BOUNDARIES.md:
- modules 与 agent 的依赖方向
- modules 之间的依赖方向
- integrations 的使用边界

NO_GO_RULES.md:
- 不允许 Agent 直连数据库
- 不允许 LLM 决定 user_id
- 不允许未确认草稿入库
- 不允许医学诊断和处方建议

GIT_WORKFLOW.md:
- 分支规范
- commit message 规范
- PR 检查清单
```

Task 00.3 创建基础 README

根目录 README 必须说明：

```
项目定位
技术栈
目录结构
本地启动方式
开发顺序
医疗安全边界
```

验收标准

```
目录结构完整。
README 能解释项目定位。
docs/architecture 下有工程规则。
Codex 看到文档后知道不能乱改架构。
```


Phase 01: 后端基础设施

目标

让后端能启动、能连接数据库、能跑迁移、能统一配置。

Task 01.1 初始化 FastAPI 后端

创建文件：

```
backend/app/main.py
backend/app/api/router.py
backend/app/core/config.py
backend/app/core/logging.py
backend/app/core/exceptions.py
backend/app/core/security.py
backend/app/core/constants.py
```

要求：

```
FastAPI app 可启动。
有统一 API router。
有全局异常处理。
有 CORS 配置。
有结构化日志。
有 /health 接口。
```

接口：

```
GET /health
```

返回：

```
{
  "status": "ok",
  "service": "family-health-agent"
}
```

Task 01.2 配置 Settings

backend/app/core/config.py 至少包含：

```
APP_NAME
ENV
DEBUG
DATABASE_URL
REDIS_URL
```

```
SECRET_KEY
ACCESS_TOKEN_EXPIRE_MINUTES
LLM_PROVIDER
OPENAI_API_KEY
LOCAL_STORAGE_DIR
S3_ENDPOINT
S3_BUCKET
DAILY_REPORT_HOUR
```

要求：

使用 pydantic-settings。
.env 文件加载。
测试环境可以覆盖配置。
不能把密钥写死在代码里。

Task 01.3 配置 SQLAlchemy

创建：

```
backend/app/db/session.py
backend/app/db/base.py
backend/app/db/init_db.py
backend/app/db/transaction.py
backend/app/db/mixins.py
```

要求：

使用 SQLAlchemy 2.x 风格。
统一 UUID 主键。
统一 created_at / updated_at。
需要软删除的表预留 deleted_at。
Session 通过 FastAPI Depends 注入。
事务封装统一管理 commit/rollback。

Task 01.4 配置 Alembic

创建：

```
backend/alembic/
backend/alembic.ini
```

要求：

alembic revision --autogenerate 可用。
alembic upgrade head 可用。
能自动读取 app/db/base.py 中的 metadata。

Task 01.5 配置 Docker Compose

创建或补齐：

```
docker-compose.dev.yml
infra/postgres/init.sql
infra/redis/.gitkeep
infra/minio/.gitkeep
```

开发环境服务：

```
backend
postgres
redis
minio
```

第一阶段 Redis / MinIO 可暂时不用，但容器和配置位置必须存在。

验收标准

```
docker-compose -f docker-compose.dev.yml up 成功。
GET /health 返回 ok。
backend 能连接 PostgreSQL。
alembic current 正常。
pytest 能启动空测试。
```

Phase 02: 数据库模型与 Alembic

目标

把长期健康系统的数据地基打好。健康产品不能先写 Agent，再补数据库。

通用建模规则

所有表使用 UUID 主键。
所有健康事实表必须有 user_id。
所有家庭共享相关表必须有 family_id。
所有时间序列数据必须有 measured_at / event_date / report_date 等时间字段。
所有 Agent trace 表必须能通过 request_id 串起来。
所有敏感访问必须可审计。

Task 02.1 identity 模块模型

路径：

```
backend/app/modules/identity/models.py
```

表：

```
users
user_auth_accounts
login_sessions
refresh_tokens
```

users 字段：

```
id
phone
email
password_hash
nickname
avatar_url
gender
birth_date
status
created_at
updated_at
last_login_at
```

user_auth_accounts 字段：

```
id
user_id
```

```
provider
provider_user_id
union_id
created_at
```

索引要求：

```
users.phone unique nullable
users.email unique nullable
user_auth_accounts(provider, provider_user_id) unique
```

Task 02.2 family 模块模型

路径：

```
backend/app/modules/family/models.py
```

表：

```
families
family_members
family_invitations
```

families:

```
id
name
owner_user_id
created_at
updated_at
```

family_members:

```
id
family_id
user_id
role
relationship_label
display_name
status
joined_at
created_at
updated_at
```

关键约束：

一个 user 可以加入多个 family。
一个 family 可以包含多个 user。
同一个 family 内同一个 user 只能 active 一次。
relationship_label 是家庭内关系标签，不是用户全局身份。

Task 02.3 permissions 模块模型

路径:

```
backend/app/modules/permissions/models.py
```

表:

```
member_share_permissions  
permission_audit_logs
```

member_share_permissions:

```
id  
family_id  
user_id  
share_all  
can_view_profile  
can_view_metrics  
can_view_reports  
can_view_symptoms  
can_view_medical_events  
can_view_documents  
can_view_alerts  
can_create_symptom_records  
can_create_metric_records  
can_upload_documents  
can_create_medical_events  
can_generate_reports  
can_generate_doctor_visit_summary  
can_export_summary  
updated_at
```

关键约束:

```
权限是 target_user_id 对 family 的共享设置。  
不是 current_user 想看就能看。  
权限不足时不得透露数据是否存在。
```

Task 02.4 health_profile 模块模型

路径:

```
backend/app/modules/health_profile/models.py
```

表:

```
health_profiles
```

字段:

```
id
user_id
height_cm
gender
birth_date
blood_type
health_goal
chronic_conditions_summary
allergy_summary
medication_summary
created_at
updated_at
```

Task 02.5 health_data 模块模型

路径：

```
backend/app/modules/health_data/models.py
```

表：

```
health_metrics
blood_pressure_records
health_data_import_jobs
```

health_metrics：

```
id
user_id
metric_type
value_numeric
value_text
unit
measured_at
period_start
period_end
source
source_detail
confidence_level
note
created_at
updated_at
```

blood_pressure_records：

```
id
user_id
systolic
diastolic
pulse
measured_at
measurement_context
arm
posture
source
confidence_level
```

```
note
created_at
updated_at
```

索引要求：

```
health_metrics(user_id, metric_type, measured_at)
blood_pressure_records(user_id, measured_at)
```

Task 02.6 health_record 模块模型

路径：

```
backend/app/modules/health_record/models.py
```

表：

```
symptom_records
health_record_drafts
```

symptom_records:

```
id
user_id
family_id
created_by_user_id
raw_text
symptom_name
body_part
severity
started_at
ended_at
duration_text
possible_trigger
related_metric_types
action_taken
follow_up_needed
follow_up_at
status
confidence_level
ai_summary
timeline_visible
source
created_at
updated_at
```

health_record_drafts 用于 AI 草稿确认：

```
id
user_id
family_id
created_by_user_id
raw_text
draft_json
status
expires_at
```

```
confirmed_at
created_at
updated_at
```

Task 02.7 medical_timeline 模块模型

路径：

```
backend/app/modules/medical_timeline/models.py
```

表：

```
medical_events
medication_records
allergy_records
follow_up_items
```

medical_events：

```
id
user_id
family_id
created_by_user_id
event_type
title
event_date
event_date_text
hospital_or_org
department
diagnosis_text
summary
doctor_advice
medications
key_findings
follow_up_needed
follow_up_at
related_document_id
source
confidence_level
timeline_visible
status
created_at
updated_at
```

重要规则：

```
diagnosis_text 不能由 AI 自行生成。
diagnosis_text 只能来自用户确认的医生诊断、医疗资料或手动填写。
```

Task 02.8 document_center / document_processing 模型

路径：

```
backend/app/modules/document_center/models.py
backend/app/modules/document_processing/models.py
```

表：

```
medical_documents
document_versions
document_access_logs
document_processing_jobs
document_extraction_results
medical_event_drafts
```

medical_documents：

```
id
user_id
family_id
uploaded_by_user_id
document_type
title
file_name
file_path
file_mime_type
file_size
document_date
document_date_text
hospital_or_org
description
ai_extract_status
ai_summary
extracted_json
confirmed_at
related_event_count
visibility
created_at
updated_at
```

安全要求：

Agent 工具不得返回 file_path。
文件读取必须通过 document_center / document_processing 服务。

Task 02.9 reports / alerts 模型

路径：

```
backend/app/modules/reports/models.py
backend/app/modules/alerts/models.py
```

表：

```
daily_reports
weekly_reports
family_reports
report_generation_logs
```

```
alerts
alert_rules
alert_events
alert_delivery_logs
```

daily_reports:

```
id
user_id
family_id
report_date
overall_status
status_level
summary_text
highlights
concerns
suggestions
metrics_snapshot
related_symptom_record_ids
related_alert_ids
generated_by
generation_status
created_at
updated_at
```

alerts:

```
id
user_id
family_id
created_by_user_id
alert_type
level
title
message
suggested_action
related_entity_type
related_entity_id
trigger_reason
status
due_at
resolved_at
source
created_at
updated_at
```

Task 02.10 agent / audit 模型

路径:

```
backend/app/agent/harness/tracing.py
backend/app/agent/harness/memory.py
backend/app/modules/audit/models.py
```

表:

```
agent_traces
agent_tool_calls
agent_safety_checks
agent_memories
audit_logs
data_access_logs
privacy_events
```

agent_traces:

```
id
request_id
session_id
workflow_name
trigger_type
current_user_id
current_family_id
target_user_id
source_page
raw_input_summary
final_output_summary
status
error_type
error_message
started_at
ended_at
duration_ms
created_at
```

agent_tool_calls:

```
id
request_id
workflow_name
tool_name
access_mode
risk_level
current_user_id
target_user_id
permission_checked
permission_result
input_summary
output_summary
status
error_type
error_message
duration_ms
created_at
```

验收标准

alembic upgrade head 成功。
所有核心表创建成功。
核心外键关系正确。
user_id / family_id / measured_at / report_date 等关键字段有索引。

没有循环依赖导致 migration 失败。
pytest 能创建测试数据库并迁移。

Phase 03: Seed Demo Data

目标

开发任何 Agent 前，必须有稳定演示数据。没有数据就开发 Agent，只会造成假回答和假闭环。

Task 03.1 创建 demo 用户

路径：

```
backend/scripts/seed_demo_data.py
```

创建：

```
Gala  
爸爸  
妈妈
```

Task 03.2 创建 demo 家庭

家庭名：Gala 的家庭

成员：

- Gala owner relationship_label=本人
- 爸爸 member relationship_label=爸爸
- 妈妈 member relationship_label=妈妈

Task 03.3 创建权限

默认：

```
share_all = true  
can_view_profile = true  
can_view_metrics = true  
can_view_reports = true  
can_view_symptoms = true  
can_view_medical_events = true  
can_view_documents = true  
can_view_alerts = true
```

Task 03.4 创建健康数据

爸爸：

最近 30 天 10 条血压记录
其中近 3 次略偏高
2 条心率记录
1 条体检事件
1 条复查提醒

妈妈：

2 条膝盖疼症状记录
7 天步数记录
1 条日报

Gala：

7 天睡眠记录
7 天步数记录
3 条体重记录
3 条心率记录
1 条日报

Task 03.5 seed 可重复执行

要求：

重复运行不会无限插入重复用户。
可以使用固定 demo phone/email。
已有 demo 数据时先清理 demo namespace 或 upsert。

验收标准

运行 seed_demo_data.py 可重复执行，不产生脏数据。
数据库中有完整家庭和健康数据。
后续 Agent 查询有数据可查。

Phase 04: 核心业务模块 Service 层

目标

先把非 AI 的业务系统跑通。健康系统本身必须成立，Agent 只是上层能力。

通用规则

service 可以调用 repository。
service 不直接调用 LLM。
service 不依赖 LangGraph。
service 返回结构化数据，不返回自然语言长回答。
service 内部必须处理权限相关业务前置条件，但 Agent 权限仍由 Harness 管。

Task 04.1 identity_service

路径：

```
backend/app/modules/identity/service.py
```

实现：

```
create_user  
get_user_by_id  
get_user_by_phone  
get_user_by_email  
verify_password  
update_profile  
disable_user
```

Task 04.2 family_service

路径：

```
backend/app/modules/family/service.py
```

实现：

```
create_family  
invite_member  
join_family  
list_my_families  
list_family_members
```

```
get_family_member
resolve_member_reference
```

resolve_member_reference 要支持：

```
我 / 本人 / 我自己
爸爸 / 我爸 / 父亲
妈妈 / 我妈 / 母亲
老婆 / 妻子 / 配偶
老公 / 丈夫 / 配偶
孩子 / 儿子 / 女儿
display_name
nickname
```

Task 04.3 permission_service

路径：

```
backend/app/modules/permissions/service.py
```

实现：

```
check_member_permission
update_share_permission
get_member_permission
assert_same_family
require_family_membership
```

要求：

本人访问本人允许。
家人访问必须同一 family。
share_all=true 时允许默认共享。
documents 权限必须单独保留检查。
权限不足不能透露数据是否存在。

Task 04.4 health_data_service

路径：

```
backend/app/modules/health_data/service.py
```

实现：

```
create_metric
get_recent_metrics
get_metric_summary
create_blood_pressure
get_recent_blood_pressure
get_blood_pressure_summary
calculate_data_quality
```

summary 输出至少包含：

```
count
latest_value
latest_measured_at
average_value
min_value
max_value
trend
data_quality
```

血压 summary 输出：

```
count
latest_systolic
latest_diastolic
latest_pulse
avg_systolic
avg_diastolic
max_systolic
max_diastolic
trend
data_quality
```

Task 04.5 health_record_service

路径：

```
backend/app/modules/health_record/service.py
```

实现：

```
create_symptom_record
create_symptom_record_draft
save_symptom_record_from_draft
get_symptom_records
get_symptom_summary
```

注意：

```
create_symptom_record_draft 只生成草稿。
save_symptom_record_from_draft 才正式写入。
正式写入必须有用户确认。
```

Task 04.6 medical_timeline_service

路径：

```
backend/app/modules/medical_timeline/service.py
```

实现：

```
create_medical_event  
get_medical_events  
get_timeline  
get_follow_up_items  
link_document
```

Task 04.7 document_center_service

路径:

```
backend/app/modules/document_center/service.py
```

实现:

```
upload_document_metadata  
get_document  
list_documents  
update_extract_status  
link_events  
get_safe_document_summary
```

要求:

```
list/get 给 Agent 用时不能返回真实 file_path。  
file_path 只允许内部服务使用。
```

Task 04.8 reports_service

路径:

```
backend/app/modules/reports/service.py
```

实现:

```
get_latest_daily_report  
get_daily_report_by_date  
save_daily_report  
check_existing_report  
list_reports
```

Task 04.9 alerts_service

路径:

```
backend/app/modules/alerts/service.py
```

实现:

```
create_alert  
get_active_alerts  
resolve_alert  
mark_read  
list_alerts
```

验收标准

所有 service 有单元测试。
service 不依赖 LLM。
service 不依赖 LangGraph。
service 可被 API 和 Agent Tools 复用。

Phase 05: 基础 API 层

目标

在没有 Agent 的情况下，健康系统也能使用。

Task 05.1 API Router

路径：

```
backend/app/api/router.py
```

注册：

```
/api/auth  
/api/users  
/api/families  
/api/permissions  
/api/health-profile  
/api/health-data  
/api/blood-pressure  
/api/health-records  
/api/medical-events  
/api/documents  
/api/reports  
/api/alerts
```

Task 05.2 Auth API

实现：

```
POST /api/auth/register  
POST /api/auth/login  
GET /api/auth/me  
POST /api/auth/logout
```

第一阶段可使用模拟登录，但接口结构必须按正式登录预留。

Task 05.3 Family API

实现：

```
POST /api/families  
GET /api/families  
GET /api/families/{family_id}
```

```
GET /api/families/{family_id}/members
POST /api/families/{family_id}/invite
POST /api/families/join
```

Task 05.4 Health Data API

实现：

```
POST /api/health-data/metrics
GET /api/health-data/metrics/recent
POST /api/blood-pressure
GET /api/blood-pressure/recent
```

Task 05.5 Health Records API

实现：

```
POST /api/health-records
GET /api/health-records
GET /api/health-records/{record_id}
```

Task 05.6 Reports / Alerts API

实现：

```
GET /api/reports/daily/latest
GET /api/reports/daily/{date}
GET /api/alerts/active
POST /api/alerts/{alert_id}/resolve
```

验收标准

可以创建用户。
可以创建家庭。
可以查询家庭成员。
可以创建血压记录。
可以创建症状记录。
可以查询日报和提醒。
所有 API 有 Pydantic schema。
API 层不写复杂业务逻辑。

Phase 06: 权限系统闭环

目标

在 Agent 之前，先证明权限系统可靠。

Task 06.1 权限检查测试

创建：

```
backend/tests/unit/modules/permissions/test_permission_service.py
```

测试场景：

本人查本人指标：允许。
子女查爸爸指标，爸爸 share_all=true：允许。
子女查爸爸 documents，can_view_documents=false：拒绝。
用户不在家庭中：拒绝。
没有 family_id 查家人：拒绝。
权限拒绝时不透露数据是否存在。

Task 06.2 API 权限接入

所有家人数据 API 必须检查：

```
current_user_id  
current_family_id  
target_user_id  
permission_type  
action
```

验收标准

test_permissions.py 全部通过。
所有家人数据 API 必须过 permission_service。
权限错误返回安全提示。
权限不足不透露数据是否存在。

Phase 07: Health Agent Harness

目标

实现 Agent 安全运行外壳。

Task 07.1 runtime.py

路径：

```
backend/app/agent/harness/runtime.py
```

实现：

```
RuntimeContext
RuntimeManager.create_context
RuntimeManager.with_target_member
RuntimeManager.mark_user_confirmed
RuntimeManager.validate_context
```

RuntimeContext 字段：

```
request_id
session_id
current_user_id
current_family_id
workflow_name
trigger_type
raw_input
source_page
target_user_id
family_member_id
target_display_name
is_system_trigger
user_confirmed_action
created_at
```

Task 07.2 permissions.py

路径：

```
backend/app/agent/harness/permissions.py
```

实现：

```
PermissionCheckInput
PermissionResult
```

```
PermissionManager.require
PermissionManager.require_many
PermissionManager.check_system_permission
```

它调用：

```
modules/permissions/service.py
```

Task 07.3 tool_registry.py

路径：

```
backend/app/agent/harness/tool_registry.py
```

实现：

```
ToolSpec
TOOL_SPECS
ToolRegistry.get_tool_spec
ToolRegistry.list_tools_for_workflow
ToolRegistry.validate_tool_call
ToolRegistry.can_expose_to_llm
```

要求：

LLM 只能看到 read 类工具。
写入工具 internal_only。
写入工具必须 user_confirmed_action=true。
workflow 不在白名单内不能调用工具。

Task 07.4 safety.py

路径：

```
backend/app/agent/harness/safety.py
```

实现：

```
SafetyManager.detect_input_risk
SafetyManager.check_output
SafetyManager.rewrite_unsafe_answer
SafetyManager.apply_medical_boundary
```

必须拦截：

诊断
开药
停药
保证没事

夸大风险
把无记录当无问题

Task 07.5 tracing.py

路径：

backend/app/agent/harness/tracing.py

实现：

```
TraceContext
TracingManager.start_trace
TracingManager.end_trace
TracingManager.record_tool_call
TracingManager.record_safety_check
sanitize_text
```

Task 07.6 errors.py

路径：

backend/app/agent/harness/errors.py

实现：

```
AgentError
ContextMissingError
TargetMemberMissingError
PermissionDeniedError
ToolNotAllowedError
ToolExecutionError
ToolTimeoutError
LLMOutputParseError
DataInsufficientError
DocumentProcessingError
SafetyBlockedError
WriteFailedError
AgentErrorResponse
ErrorManager.to_user_response
```

Task 07.7 guards.py

路径：

backend/app/agent/harness/guards.py

实现硬规则：

没有 RuntimeContext 不执行。
没有 current_user_id 不执行用户请求。
没有 target_user_id 不查健康数据。
没有 current_family_id 不查家人数据。
没有 user_confirmed_action 不执行写入。
资料工具不能返回真实文件路径。

Task 07.8 runner.py / tool_caller.py

路径：

```
backend/app/agent/harness/runner.py  
backend/app/agent/harness/tool_caller.py
```

实现：

```
HarnessRunner.run  
HarnessToolCaller.call_tool
```

验收标准

Harness 单测全部通过。
未确认不能写入。
无权限不能调用工具。
Tool call 会写 trace。
Safety 会改写危险回答。

Phase 08: Agent Tools

目标

把业务 service 包装成受控工具。

Tools 清单

必须实现：

```
resolve_family_member
check_member_permission
get_recent_metrics
get_recent_blood_pressure
get_symptom_records
get_medical_events
get_medical_documents
get_latest_daily_report
get_active_alerts
create_symptom_record_draft
save_symptom_record_from_draft
extract_document_summary
save_medical_event_from_document
generate_daily_report
```

文件路径

```
backend/app/agent/tools/member_tools.py
backend/app/agent/tools/permission_tools.py
backend/app/agent/tools/profile_tools.py
backend/app/agent/tools/metric_tools.py
backend/app/agent/tools/blood_pressure_tools.py
backend/app/agent/tools/symptom_tools.py
backend/app/agent/tools/medical_event_tools.py
backend/app/agent/tools/document_tools.py
backend/app/agent/tools/report_tools.py
backend/app/agent/tools/alert_tools.py
```

工具规则

所有 Tool 只能接收 runtime 中的 target_user_id。
Tool input 不允许包含 current_user_id。
Tool input 不允许包含自由 target_user_id。
Tool 不直接创建 DB session。

Tool 不绕过 service。
写入 Tool 必须通过 HarnessToolCaller。

验收标准

读工具可以返回结构化摘要。
写工具未确认时被拦截。
工具调用会记录 trace。
权限不足不会调用底层 service。

Phase 09: LLM Client 与 Prompt 管理

目标

统一模型调用，不让 LLM 调用散落在代码各处。

Task 09.1 LLM Client

路径：

```
backend/app/agent/llm_client.py
backend/app/integrations/llm/openai_compatible.py
backend/app/integrations/llm/mock.py
```

实现：

```
LLMClient.complete
LLMClient.structured_complete
```

要求：

测试环境默认 mock LLM。
业务代码不直接调用 OpenAI SDK。
structured output 解析失败有重试和错误处理。

Task 09.2 Prompt 管理

路径：

```
prompts/health_record/
prompts/daily_report/
prompts/document_extract/
prompts/doctor_visit/
prompts/safety/
```

Prompt 分类：

```
intent_classification
symptom_extraction
document_extraction
daily_report_expression
doctor_visit_summary
safety_rewrite
```

每个 prompt 必须包含：

name
version
purpose
input schema
output schema
safety constraints
examples

验收标准

mock LLM 能跑通测试。
真实 LLM provider 可配置。
Prompt 不散落在代码里。
所有 structured output 都有 Pydantic schema 校验。

Phase 10: LangGraph Workflows

目标

实现真正的 Agent 流程。

Task 10.1 chat_workflow

路径：

```
backend/app/agent/workflows/chat_workflow.py
```

能力：

```
意图识别
成员解析
权限检查
工具路由
回答生成
Safety
handoff 子工作流
```

支持 intent：

```
query_daily_status
query_metrics
query_blood_pressure
query_symptoms
query_medical_events
query_documents
query_alerts
create_health_record
generate_daily_report
doctor_visit_summary
health_knowledge_qa
unknown
```

Task 10.2 free_text_record_workflow

路径：

```
backend/app/agent/workflows/free_text_record_workflow.py
```

流程：

```
用户一句话
解析成员
```

权限检查
生成草稿
返回确认卡
用户确认
保存 symptom_records / metrics / blood_pressure / alerts

Task 10.3 daily_report_workflow

路径：

backend/app/agent/workflows/daily_report_workflow.py

流程：

检查已有日报
收集数据
规则分析
生成 structured_report_data
LLM 写表达
Safety
保存 daily_reports
生成 alerts

Task 10.4 document_extract_workflow

路径：

backend/app/agent/workflows/document_extract_workflow.py

流程：

保存原始资料
读取文本/OCR
提取摘要
生成 medical_event 草稿
用户确认
保存 medical_events
生成 follow_up alert

Task 10.5 doctor_visit_summary_workflow

路径：

backend/app/agent/workflows/doctor_visit_summary_workflow.py

流程：

识别就医主题
解析成员

权限检查
收集近期指标/症状/事件/提醒
生成就医摘要
生成医生沟通问题
Safety

验收标准

chat_workflow 不直接保存数据。
free_text_record 未确认不入库。
daily_report 规则算事实，LLM 写表达。
document_extract 先保存原件，再提取。
doctor_visit_summary 不诊断，不给治疗方案。

Phase 11: 日报与提醒系统

目标

让首页和家庭页有持续价值。

Task 11.1 规则服务

路径：

```
backend/app/modules/reports/daily/rules.py
backend/app/modules/alerts/rules.py
backend/app/agent/rules/daily_report_rules.py
```

规则：

```
数据不足
血压近期偏高
步数明显偏低
睡眠不足
症状持续
复查接近
资料待确认
```

Task 11.2 日报生成器

路径：

```
backend/app/modules/reports/daily/generator.py
```

日报状态：

```
normal
attention
follow_up
insufficient_data
```

Task 11.3 提醒生成

提醒类型：

```
metric_attention
symptom_follow_up
medical_follow_up
```

medication_reminder
data_missing
document_review

验收标准

一天轻微波动只进日报。
连续异常才生成 alert。
无数据生成 insufficient_data。
日报不诊断、不用药、不保证没事。

Phase 12: 文档中心与资料处理

目标

实现正式健康档案的重要入口。

实现顺序

```
document_center 上传与元信息
本地文件存储
document_processing_jobs
文本读取
OCR 适配器预留
AI 提取摘要
medical_event 草稿
用户确认
写入 medical_events
```

Task 12.1 文件上传与存储

路径：

```
backend/app/modules/document_center/storage.py
backend/app/integrations/storage/local.py
```

Task 12.2 文档处理任务

路径：

```
backend/app/modules/document_processing/pipeline.py
backend/app/modules/document_processing/tasks.py
```

Task 12.3 OCR 适配器

路径：

```
backend/app/modules/document_processing/ocr/base.py
backend/app/modules/document_processing/ocr/local_ocr.py
backend/app/modules/document_processing/ocr/cloud_ocr.py
```

Task 12.4 提取器

路径：

```
backend/app/modules/document_processing/extractors/checkup_report_extractor.py
backend/app/modules/document_processing/extractors/lab_test_extractor.py
backend/app/modules/document_processing/extractors/prescription_extractor.py
backend/app/modules/document_processing/extractors/discharge_summary_extractor.py
```

验收标准

原始资料先保存。
提取失败不影响原件保存。
AI 只生成草稿。
用户确认后才写 `medical_events`。
`medical_events` 必须关联 `related_document_id`。

Phase 13: 知识库 / RAG

目标

支持健康知识解释，但不替代医生。

实现

```
knowledge_documents  
knowledge_chunks  
knowledge_sources  
retriever  
reranker  
source_policy  
search_health_knowledge tool  
health_knowledge_qa_workflow
```

路径

```
backend/app/modules/knowledge/ingestion/  
backend/app/modules/knowledge/retrieval/  
backend/app/modules/knowledge/safety/source_policy.py
```

验收标准

```
知识回答必须引用来源。  
不能基于知识库诊断用户。  
知识库只用于解释指标、健康常识、设备说明。
```

Phase 14: 导出与分享

目标

支持就医摘要和健康档案导出。

实现

```
export_jobs
export_files
pdf_renderer
markdown_renderer
doctor_visit_package
medical_record_package
share_links
```

路径

```
backend/app/modules/export/renderers/pdf_renderer.py
backend/app/modules/export/renderers/markdown_renderer.py
backend/app/modules/export/packages/doctor_visit_package.py
backend/app/modules/export/packages/medical_record_package.py
```

验收标准

导出必须检查权限。
导出内容必须标注数据来源。
就医摘要必须包含“不替代医生诊断”。
分享链接可撤销。

Phase 15: Web 前端

目标

把后端能力变成可用产品。

页面顺序

登录/模拟登录
首页 Dashboard
家庭页
成员详情页
健康档案页
血压记录页
健康随手记页
确认卡组件
日报页
提醒页
资料页
AI 管家页
设置与权限页

页面模块路径

apps/web/app/dashboard/
apps/web/app/fami ly/
apps/web/app/members/
apps/web/app/records/
apps/web/app/documents/
apps/web/app/reports/
apps/web/app/alerts/
apps/web/app/agent/
apps/web/app/settings/
apps/web/components/

验收标准

用户 5 秒内能看懂今日状态。
记录症状不超过 10 秒。
记录血压不超过 15 秒。
AI 草稿必须有确认卡。
权限不足时前端显示安全提示。

Phase 16: 测试体系

目标

防止豆腐渣工程。

测试目录

```
backend/tests/unit/  
backend/tests/integration/  
backend/tests/e2e/  
backend/tests/fixtures/
```

必须测试

权限测试
工具门禁测试
Safety 测试
Workflow 测试
Service 测试
API 测试
数据库迁移测试
前端核心流程测试

关键测试用例

无权限不能查家人资料。
LLM 不能调用写入工具。
未确认不能保存草稿。
高风险症状必须提示就医。
无数据不能说没问题。
系统内无记录不能推断现实没有。
document 工具不能返回真实文件路径。
日报不能生成诊断。
就医摘要不能给治疗方案。

验收标准

pytest 全部通过。
核心安全测试必须阻断失败合并。

每个核心 service 有单测。
每个 workflow 有集成测试。

Phase 17: 部署与运维

目标

让项目可部署、可观察、可恢复。

实现

```
Dockerfile
docker-compose.prod.yml
Nginx
PostgreSQL backup
MinIO storage
Redis
Sentry
OpenTelemetry 预留
日志轮转
健康检查
```

验收标准

```
一条命令启动 dev 环境。
生产配置不使用 debug。
数据库有备份方案。
文件存储有迁移方案。
Agent trace 可查看。
错误日志可定位。
```

Phase 18: 移动端 / 设备 / 通知扩展

目标

在成品架构上继续扩展，而不是推倒重来。

移动端

apps/mobile
Expo / React Native
复用 packages/api-client
复用 shared-types

设备接入

Apple Health
Google Fit
华为健康
小米健康
体脂秤
血压计

通知

App Push
微信服务通知
短信
邮件
站内通知

验收标准

设备数据进入 normalized_device_data。
设备数据再进入 health_metrics / blood_pressure_records。
通知由 alerts 驱动。
移动端不绕过后端权限。

最终完成标准

当以下条件全部满足，项目才算真正落地：

1. 用户可以注册/登录。
2. 用户可以创建家庭并加入多个家庭。
3. 每个家庭成员都有自己的健康档案。
4. 家庭权限可控制不同数据类型共享。
5. 可以记录睡眠、步数、体重、BMI、心率、血压。
6. 可以一句话记录健康随手记。
7. AI 可以生成草稿，用户确认后保存。
8. 可以上传健康资料并生成结构化事件草稿。
9. 可以保存正式医疗事件和复查提醒。
10. 系统可以生成每日健康简报。
11. 家庭页可以查看成员状态卡。
12. AI 管家可以查询指标、症状、事件、资料、提醒。
13. 就医前可以生成摘要。
14. 所有 Agent 查询都有权限检查。
15. 所有写入都需要用户确认。
16. 所有健康回答都有 Safety。
17. 所有关键 Agent 执行都有 Trace。
18. 前端可以完成核心闭环。
19. 测试覆盖核心安全逻辑。
20. 项目可以 Docker 部署。

给 Codex 的总启动 Prompt

第一次交给 Codex 时，使用下面这段，不要让它自由发挥：

你正在开发 family-health-agent 项目。

这是一个成品级模块化单体架构的家庭健康 Agent 系统。
请严格阅读并遵守：

1. docs/architecture/家庭健康Agent_项目架构设计_v1.0.md
2. CODEX_IMPLEMENTATION_PLAN.md
3. docs/architecture/MODULE_BOUNDARIES.md
4. docs/architecture/NO_GO_RULES.md

你不能重构顶层目录。

你不能把业务逻辑写进 API 层。

你不能让 Agent 直接访问数据库。

你不能让 LLM 决定 user_id、family_id、target_user_id。

你不能绕过权限系统。

你不能实现医学诊断、处方建议、药物剂量建议。

你不能让未确认 AI 草稿入库。

现在只执行 Phase 00 和 Phase 01。

完成后输出：

1. 你创建/修改了哪些文件。
2. 如何启动项目。
3. 如何验证 /health。
4. 哪些任务还没有做。

不要提前实现后续 Phase。

每个 Phase 完成后的固定汇报格式

要求 Codex 每次完成一个 Phase 后按这个格式回复：

```
## 完成阶段
Phase XX: <阶段名>

## 创建/修改文件
- path/to/file1
- path/to/file2

## 实现内容
- ...

## 未实现内容
- ...

## 验收方式
- 命令：...
- 预期结果：...

## 风险与注意事项
- ...

## 下一阶段建议
Phase XX+1: ...
```

这样你可以逐步验收，而不是让 Codex 一口气乱做完整项目。